

Coding Standards — The Outliers

Bộ Quy tắc Lập trình Chung

Ngô Tấn Đạt

05/06/2026

Bộ quy tắc chung áp dụng cho toàn bộ team. Mọi người đọc và làm theo trước khi commit code.

Mục lục

1 Đặt tên (Naming)	2
2 Format Code	2
3 Import	2
4 Hàm và Class	2
5 Xử lý lỗi	3
6 Không lặp code (DRY)	3
7 Jupyter Notebook	3
8 Commit Message	3
9 Checklist Trước Khi Commit	4
10 OOP (Tùy chọn — không bắt buộc)	4
11 Không Làm	5

1 Đặt tên (Naming)

Loại	Quy tắc	Ví dụ
Biến, hàm	snake_case	load_data, site_id
Class	PascalCase	SolarSite, WeatherLoader
Hằng số	UPPER_CASE	BUCKET_NAME, DB_HOST
File Python	snake_case	load_01_dims.py
File tài liệu	YYYY_MM_DD_content_Author.ext	2026_05_25_schema_design_TanDat.tex

Tên phải nói lên được ý nghĩa — không dùng x, df2, tmp.

2 Format Code

- Thụt lề: **4 spaces**, không dùng tab
- Độ dài dòng: tối đa **88 ký tự** (chuẩn Black)
- Dùng **Black** để auto-format trước khi commit:

```
black src/  
ruff check src/
```

3 Import

Sắp xếp theo thứ tự, cách nhau 1 dòng trắng:

```
1 # 1. Thu vien chuan  
2 import os  
3 from pathlib import Path  
4  
5 # 2. Thu vien ben thu ba  
6 import pandas as pd  
7 import psycopg2  
8  
9 # 3. Module noi bo  
10 from src.config import DB_HOST
```

4 Hàm và Class

- Mỗi hàm chỉ làm **1 việc** (Single Responsibility)
- Dùng **type hints** cho tham số và giá trị trả về
- Docstring ngắn gọn cho hàm quan trọng
- Tránh hàm dài hơn **30 dòng** — nếu dài thì tách ra

```

1 def load_csv(path: str) -> pd.DataFrame:
2     """Load raw CSV file from given path."""
3     return pd.read_csv(path)
4
5 def load_dims(conn) -> None:
6     """Load dimension tables from MinIO into PostgreSQL."""
7     ...

```

5 Xử lý lỗi

Bắt lỗi **cụ thể**, không dùng except chung chung. Không ”nuốt lỗi” im lặng.

```

1 # SAI
2 try:
3     load_data()
4 except:
5     pass
6
7 # DUNG
8 try:
9     load_data()
10 except FileNotFoundError as e:
11     print(f"File not found: {e}")
12     raise

```

6 Không lặp code (DRY)

- Logic dùng nhiều hơn 1 lần → tách thành hàm
- Không copy-paste block code giữa các file

7 Jupyter Notebook

- Đánh số cell theo thứ tự logic, chạy từ trên xuống được
- Tên notebook: 01_edu_solar_generation.ipynb
- Không để output rác (print thừa, stack trace) khi commit

8 Commit Message

Theo chuẩn Angular commit convention:

```
<type>(<scope>) [SCRUM-KEY] (<mo ta ngan>)
```

Type	Dùng khi
feat	Thêm tính năng mới
fix	Sửa lỗi
refactor	Tái cấu trúc code
chore	Công việc phụ (config, docs)

Ví dụ:

```
feat(etl)[scrum-40](add night noise filter to transform pipeline)
```

9 Checklist Trước Khi Commit

Code chạy được không lỗi

Đã chạy `black src/` và `ruff check src/`

Tên biến/hàm rõ nghĩa

Không có code thừa, print debug

Import đúng thứ tự

Commit message đúng format

10 OOP (Tùy chọn — không bắt buộc)

Dùng class khi logic **phức tạp**, có **trạng thái**, hoặc **dùng lại nhiều lần**. Với script ETL đơn giản thì hàm là đủ.

Khi nào nên dùng class

Tình huống	Dùng
Script ETL chạy 1 lần	Hàm
Connector tái sử dụng (DB, MinIO)	Class
Model ML có fit/predict	Class
Nhiều hàm dùng chung state	Class

Nguyên tắc nếu dùng class

SRP — mỗi class chỉ làm 1 việc:

```

1 # SAI - class lam qua nhieu viec
2 class DataProcessor:
3     def load(self): ...
4     def clean(self): ...
5     def train_model(self): ...
6
7 # DUNG - tach ro trach nhien
```

```

8 class DataLoader:
9     def load(self): ...
10
11 class DataCleaner:
12     def clean(self): ...

```

Dùng @property thay getter/setter thủ công:

```

1 class SolarSite:
2     def __init__(self, capacity_kw: float):
3         self._capacity_kw = capacity_kw
4
5     @property
6     def capacity_kw(self) -> float:
7         return self._capacity_kw

```

Ưu tiên **composition** hơn inheritance:

```

1 class ETLPipeline:
2     def __init__(self):
3         self.loader = DataLoader()
4         self.cleaner = DataCleaner()

```

Quy tắc underscore: _var cho internal, __var khi cần name mangling.

11 Không Làm

- Không commit trực tiếp lên main
- Không push file .env, .venv, data CSV
- Không dùng import *
- Không hardcode credential trong code